Exercise2_1

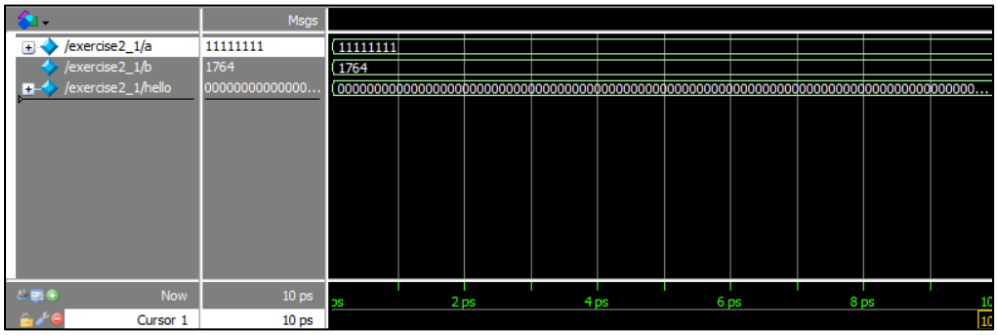
HDL Code

```
module exercise2_1;

        reg[7:0] a;
        real b;
        reg[159:0] hello, con;

        initial begin
                a=8'hFF;
                b=1764;
                hello="Hello";

                $display("Hexadecimal Value:%0h\nReal Number:%0e",a,b);
                $display("Backslash:\\\nPercent:%%\nNew tab:\tText after tab\nDouble quote:\"\"");
                $display("String:%0s",hello);

                if(a>b)
                        con = "a is greater than b";
                else
                        con = "b is greater than a";

                $display("Conditional:%0s",con);

        end

endmodule
```

Transcript

```
VSIM 66> run
# GetModuleFileName: The specified module could not be found.
#
#
# Hexadecimal Value:ff
# Real Number:1.764000e+03
# Backslash:\
# Percent:%
# New tab: Text after tab
# Double quote:""
# String:Hello
# Conditional:b is greater than a
```

Wave



Exercise2_2

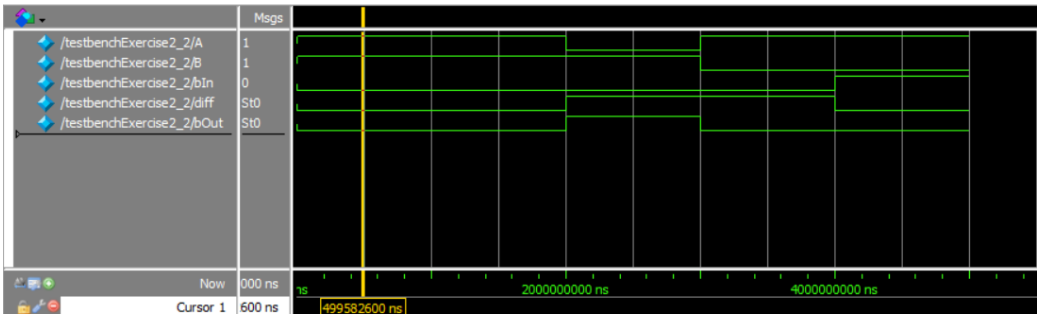
HDL Code

```
`timescale 1s / 100 ns
`include "../Module Instantation and Test Benches/drilll_2.v"

module testbenchExercise2_2;

        reg A,B,bIn;
        wire diff,bOut;

        full_subtract fs(diff,bOut,A,B,bIn);

        initial begin
                $display("A\tB\tBorrowIn\t\tDifference\t\tBorrowOut\t\tTime(s)");
                $monitor("%b\t%b\t%b\t\t%b\t\t%b\t\t%0d", A, B, bIn, diff, bOut, $time);

                A = 1'b1; B = 1'b1; bIn = 1'b0;
                #1 A = 1'b1;
                #1 A = 1'b0; B = 1'b1;
                #1 A = 1'b1; B = 1'b0;
                #1 bIn = 1'b1;
                #1 $finish;
        end

endmodule
```

Transcript

```
VSIM 26> run
# GetModuleFileName: The specified module could not be found.
#
#
# A     B       BorrowIn        Difference      BorrowOut       Time(s)
# 1     1       0               0               0               0
# 0     1       0               1               1               2
# 1     0       0               1               0               3
# 1     0       1               0               0               4
# ** Note: $finish    : C:/intelFPGA_lite/20.1/Verilog HDL Syntax and Semantics/Exercise2_2.v(20)
#    Time: 5 sec  Iteration: 0  Instance: /testbenchExercise2_2
# 1
# Break in Module testbenchExercise2_2 at C:/intelFPGA_lite/20.1/Verilog HDL Syntax and Semantics/Exercise2_2.v line 20
```

Wave



Exercise2_3

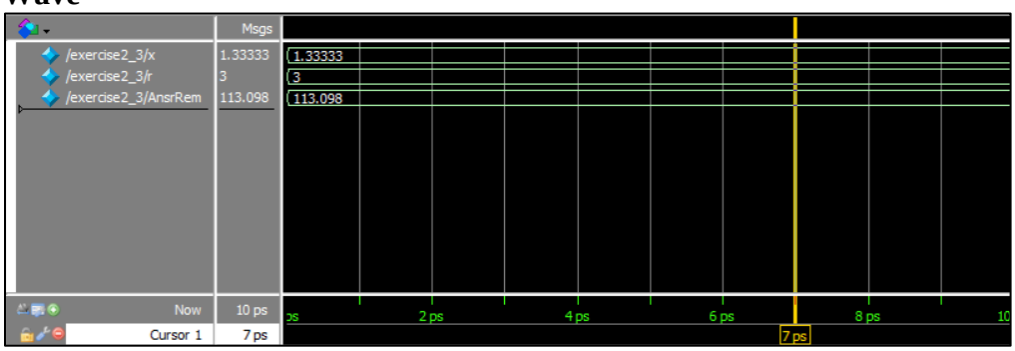
HDL Code

```
`define pi 3.1416

module exercise2_3;

        real x, r, AnsrRem;

        initial begin
                r = 3;
                x = 4.0/3.0;
                AnsrRem = x*`pi*(r*r*r);

                $display("The volume of the cylinder is, %0f cubic units.",AnsrRem);
        end

endmodule
```

Transcript

```
VSIM 43> run
# GetModuleFileName: The specified module could not be found.
#
#
# The volume of the cylinder is, 113.097600 cubic units.
```

Wave



Drill2_1

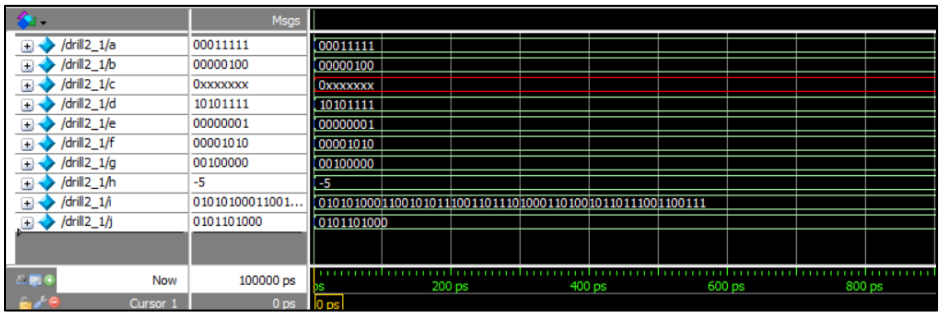
HDL Code

```
module drill2_1;

        reg [7:0] a, b, c, d, e, f, g;
        reg signed [7:0] h;
        reg [55:0] i;
        reg [9:0] j;

        initial begin
                a=5'O37;
                b=3'D4;
                c=7'hx;
                d=8'hAF;
                e=10'b01;
                f=4'd10;
                g=32;
                h=-5;
                i="Testing";
                j=3.6E2;
                $write(" %s %o %b %b %h %b",i,a,b,c,d,e);
                $write(" %d %d %d %d",f,g,h,j);
        end

endmodule
```

Transcript

```
VSIM 22> run
# GetModuleFileName: The specified module could not be found.
#
#
 Testing 037 00000100 0xxxxxxx af 00000001  10  32   -5  360
```

Wave



Drill2_2

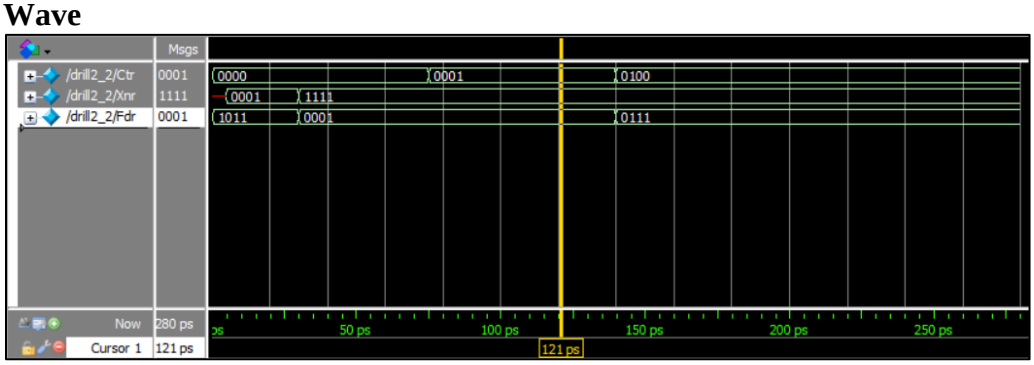
HDL Code

```
module drill2_2;

        reg [3:0]Ctr, Xnr, Fdr;

        initial begin
                Ctr=4'd0; Fdr=4'd11;
                #5 Xnr = (Ctr !=0) ? (Ctr *+ 1) : ^Fdr;
                #10 $display(Xnr);
                #15 Xnr=-9%2; Fdr=52<8'hFF;
                #20 $display(Xnr," ",Fdr);
                #25 Ctr=Xnr&&Fdr; Fdr=Xnr&Fdr;
                #30 $display(Ctr,Fdr);
                #35 Ctr=Ctr<<2; Fdr=Xnr>>1;
                #40 $display(Ctr, Fdr);
                #100 $finish;
        end

endmodule
```

Transcript

```
VSIM 71> run
#  1
# 15  1
#  1 1
#  4 7
# ** Note: $finish    : C:/intelFPGA_lite/20.1/Verilog HDL Syntax and Semantics/Drill2_2.v(15)
#    Time: 280 ps  Iteration: 0  Instance: /drill2_2
# 1
# Break in Module drill2_2 at C:/intelFPGA_lite/20.1/Verilog HDL Syntax and Semantics/Drill2_2.v line 15
```

Wave



Drill2_3

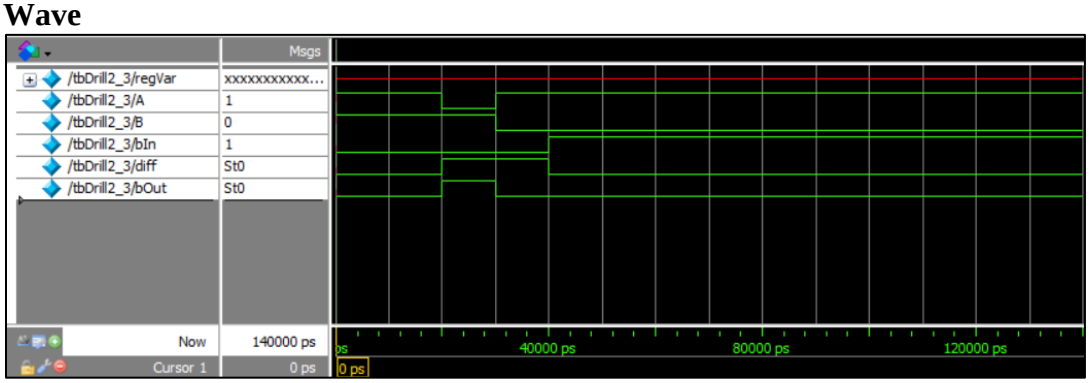
HDL Code

```
`timescale 10 ns / 100 ps
`define SIZE 8
`define STOP $finish
`define REGME reg [8*31:0]
`ifdef TEST_1
        `include "../Module Instantation and Test Benches/drill1_1.v"
`else
        `include "../Module Instantation and Test Benches/drill1_2.v"
`endif

module tbDrill2_3;
        `REGME  regVar;

        reg A,B,bIn;
        wire diff,bOut;

        full_subtract fs(diff,bOut,A,B,bIn);

        initial begin
                $display("A\tB\tBorrowIn\t\tDifference\t\tBorrowOut\t\tTime(s)");
                $monitor("%b\t%b\t%b\t\t%b\t\t%b\t\t%0d", A, B, bIn, diff, bOut, $time);

                A = 1'b1; B = 1'b1; bIn = 1'b0;
                #1 A = 1'b1;
                #1 A = 1'b0; B = 1'b1;
                #1 A = 1'b1; B = 1'b0;
                #1 bIn = 1'b1;

                #10 `STOP;
        end
endmodule
```

Transcript

```
VSIM 86> run
# GetModuleFileName: The specified module could not be found.
#
#
# A     B       BorrowIn        Difference      BorrowOut       Time(s)
# 1     1       0               0               0               0
# 0     1       0               1               1               2
# 1     0       0               1               0               3
# 1     0       1               0               0               4
# ** Note: $finish    : C:/intelFPGA_lite/20.1/Verilog HDL Syntax and Semantics/Drill2_3.v(29)
#    Time: 140 ns  Iteration: 0  Instance: /tbDrill2_3
# 1
# Break in Module tbDrill2_3 at C:/intelFPGA_lite/20.1/Verilog HDL Syntax and Semantics/Drill2_3.v line 29
```

Wave



Descriptive Explanation

Drill2_1

The drill2_1 module demonstrates various data types and formats in Verilog. It initializes registers with octal, decimal, hexadecimal, binary, and string values. The $write statement prints these values in different formats, including octal, binary, hexadecimal, and decimal, showcasing how to work with signed and unsigned integers. This is a basic example of handling multiple data formats and displaying them in a simulation.

Drill2_2

The drill2_2 module demonstrates conditional, arithmetic, bitwise, and shift operations in Verilog. Registers Ctr, Xnr, and Fdr are initialized and manipulated using conditional expressions, modulo, bitwise AND, and shifts. Values are displayed at specified time intervals with delays. The module is useful for practicing Verilog's logic and bitwise operations.

Drill2_3

This Verilog code tests a full subtractor using a conditional compilation and module instantiation. The tbDrill2_3 module sets up a testbench for the full_subtract module, which takes inputs A, B, and bIn (borrow-in) and produces outputs diff (difference) and bOut (borrow-out). The simulation runs several test cases by changing the input values and monitors the results, displaying them along with the time in the format specified. The test concludes with a STOP directive, which calls $finish. Conditional includes based on the TEST_1 flag determine whether drill1_1.v or drill1_2.v is used.

Exercise2_1

This Verilog module simulates simple operations using different data types such as an 8-bit hexadecimal, a real number, and strings. It demonstrates formatted output using $display to show values with escape characters like backslashes, percentages, tabs, and quotes. A conditional check compares the hex value a and real number b, assigning a string to con based on the comparison. The result is then displayed.

Exercise2_2

This Verilog testbench simulates a full subtractor module (full_subtract) to observe its behavior with different input combinations. The testbench defines three inputs: A, B, and bIn (borrow in), and monitors two outputs: diff (difference) and bOut (borrow out). The $display and $monitor statements provide formatted output showing the inputs and outputs with time. The test applies various combinations of A, B, and bIn to observe the corresponding difference and borrow-out values, finishing after testing several cases.

Exercise2_3

This Verilog module calculates the volume of a sphere using a defined constant pi. The module defines three real variables: x, r, and AnsrRem. It assigns the value 3 to the radius r, and the ratio 4/3 to x. The volume of the sphere is calculated using the formula $V = (4/3)\pi r^3$ and stored in AnsrRem. Finally, it prints the result using the $display function. The output will display the volume of a sphere with radius 3 in cubic units.